

LAPORAN PRATIKUM
ALGORITMA DAN PEMROGRAMAN

Single Linked List (SLL)

Disusun Oleh:

Khairun Nisa

2511532015

Dosen Pengampu:

DR. WAHYUDI, S.T, M.T

Asisten Pratikum:

Sherly Sukmadira



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas terselesaikannya laporan praktikum Algoritma dan Pemrograman ini. Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilaksanakan pada tanggal 4 Mei 2026. Melalui laporan ini, penulis berharap dapat memberikan gambaran yang jelas mengenai Single Linked List (SLL).

Penulis mengucapkan terima kasih kepada Bapak Dosen serta asisten laboratorium yang telah membimbing selama praktikum berlangsung. Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari kata sempurna, baik dari segi penyusunan, bahasa, maupun kedalaman materi. Oleh karena itu, penulis dengan rendah hati menerima segala kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi untuk perkembangan ilmu pengetahuan di teknologi informasi.

Padang, 5 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Pratikum	1
1.3 Manfaat Pratikum	2
BAB II PEMBAHASAN	3
2.1 NodeSLL.....	3
2.2 TambahSLL	3
2.3 PencarianSLL.....	6
2.4 HapusSLL	8
BAB III KESIMPULAN	12
3.1 Kesimpulan	12
DAFTAR PUSTAKA	13

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman, pengelolaan data secara efisien merupakan kebutuhan dasar. Salah satu struktur data yang paling umum digunakan adalah array, namun array memiliki keterbatasan utama: ukurannya bersifat statis (tetap) sejak awal dideklarasikan, sehingga proses penambahan atau penghapusan data di tengah-tengah sering kali memerlukan penggeseran elemen yang tidak efisien. Untuk mengatasi hal tersebut, dikembangkanlah struktur data dinamis bernama Single Linked List (Daftar Berantai Tunggal) [1].

Single Linked List bekerja dengan menyimpan data dalam unit-unit yang disebut node. Setiap node terdiri dari dua bagian: data itu sendiri dan sebuah pointer (referensi memori) yang menunjuk ke node berikutnya. Karena sifatnya yang dinamis, linked list tidak memerlukan alokasi ukuran tetap di awal. Penambahan maupun penghapusan node cukup dilakukan dengan mengatur ulang arah pointer, tanpa perlu menggeser seluruh data [2]. Sifat ini sangat cocok diterapkan pada sistem yang bekerja dengan prinsip First In, First Out (FIFO), seperti antrian pendaftaran pasien di rumah sakit. Pasien yang datang pertama akan menjadi node pertama, dan ketika dipanggil, node tersebut dihapus dari depan rantai tanpa mengganggu urutan pasien di belakangnya.

1.2 Tujuan Praktikum

1. Memahami konsep dasar Single Linked List, termasuk struktur node, pointer next, dan cara kerja rantai referensi antar elemen data.
2. Mengimplementasi operasi manipulasi linked list
3. Menerapkan prinsip FIFO
4. Melatih kemampuan pemecahan masalah

1.3 Manfaat Praktikum

1. Memahami perbedaan mendasar antara struktur data statis (array) dan dinamis (linked list) serta kapan masing-masing tepat digunakan.
2. Meningkatkan kemampuan debugging dan analisis alur program, terutama dalam melacak perubahan pointer saat operasi tambah/hapus node.
3. Membiasakan diri dengan standar penulisan kode akademik dan kolaborasi melalui GitHub, kompetensi yang sangat dibutuhkan di industri perangkat lunak.

BAB II

PEMBAHASAN

2.1 NodeSLL

```
1 package pekan5_2511532015;
2
3 public class NodeSLL_2511532015 {
4     //node bagian data
5     int data_2015;
6     //pointer ke node berikutnya
7     NodeSLL_2511532015 next_2015;
8     //konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511532015(int data) {
10         this.data_2015= data;
11         this.next_2015= null;
12     }
13 }
14
```

Gambar 2.1.1: Kode program NodeSLL_2511532015

Kelas NodeSLL_2511532015 dirancang sebagai representasi dari sebuah simpul (node) dalam struktur Single Linked List. Terdiri dari dua komponen utama:

- data_2015: bertipe integer untuk menyimpan alamat node berikutnya.
- next_2015: yang berfungsi sebagai referensi simpul berikutnya.

Konstruktor kelas menginisialisasi node dengan nilai data tertentu dan secara default mengatur referensi next menjadi null agar node tersebut belum terhubung ke node lain (masih null) saat pertama kali dibuat. Kelas ini dirancang sebagai modul terpisah yang dapat digunakan kembali, dimana kelas ini dapat diimpor dan diinstalasi di dalam kelas lain untuk membentuk struktur data linked list.

2.2 TambahSLL

```
package pekan5_2511532015;

public class TambahSLL_2511532015 {
    public static NodeSLL_2511532015 insertAtFront
(NodeSLL_2511532015 head_2015, int value_2015) {
        NodeSLL_2511532015 new_node_2015 = new NodeSLL_2511532015
(value_2015);
        new_node_2015.next_2015 = head_2015;
        return new_node_2015;
    }
}
```

```

}
//fungsi menambahkan node di akhir SLL
public static NodeSLL_2511532015 insertAtEnd
(NodeSLL_2511532015 head_2015, int value_2015) {
    //buat sebuah node dengan sebuah nilai
    NodeSLL_2511532015 newNode = new NodeSLL_2511532015
(value_2015);
    //jika list kosong maka node jadi head
    if (head_2015 == null) {
        return newNode;
    }
    //simpan head ke variabel sementara
    NodeSLL_2511532015 last= head_2015;
    //telusuri ke node akhir
    while (last.next_2015 != null) {
        last = last.next_2015;
    }
    //ubah pointer
    last.next_2015 = newNode;
    return head_2015;
}
static NodeSLL_2511532015 GetNode (int data) {
    return new NodeSLL_2511532015 (data);
}
static NodeSLL_2511532015 insertPos (NodeSLL_2511532015
headNode_2015, int position_2015, int value_2015) {
    NodeSLL_2511532015 head_2015 = headNode_2015;
    if (position_2015 < 1)
        System.out.print ("Invalid position");
    if (position_2015 == 1) {
        NodeSLL_2511532015 new_node = new NodeSLL_2511532015
(value_2015);
        new_node.next_2015 = head_2015;
        return new_node;
    } else {
        while (position_2015-- != 0) {
            if (position_2015 == 1) {
                NodeSLL_2511532015 newNode = GetNode
(value_2015);
                newNode.next_2015 =
headNode_2015.next_2015;
                headNode_2015.next_2015 = newNode;
                break;
            }
            headNode_2015 = headNode_2015.next_2015;
        }
        if (position_2015 != 1)
            System.out.print ("Posisi di luar
jangkauan");
    }
    return head_2015;
}
public static void printList_2015 (NodeSLL_2511532015
head_2015) {
    NodeSLL_2511532015 curr_2015= head_2015;
    while (curr_2015.next_2015 != null) {
        System.out.print (curr_2015.data_2015+"-->");
    }
}

```

```

        curr_2015 = curr_2015.next_2015;
    }
    if (curr_2015.next_2015 == null) {
        System.out.print (curr_2015.data_2015);
    }
    System.out.println();
}
public static void main (String [] args) {
    //buat linked list 2->3->5->6
    NodeSLL_2511532015 head_2015 = new NodeSLL_2511532015 (2);
    head_2015.next_2015 = new NodeSLL_2511532015 (3);
    head_2015.next_2015.next_2015 = new NodeSLL_2511532015 (5);
    head_2015.next_2015.next_2015.next_2015 = new
NodeSLL_2511532015 (6);
    //cetak list asli
    System.out.print ("Senarai berantai awal: ");
    printList_2015 (head_2015);
    //tambahkan node baru di depan
    System.out.print ("tambah 1 simpul di depan: ");
    int data_2015 = 1;
    head_2015 = insertAtFront (head_2015, data_2015);
    //cetak update list
    printList_2015 (head_2015);

    //tambahkan ode baru dibelakang
    System.out.print ("tambah 1 simpul di belakang: ");
    int data2_2015 = 7;
    head_2015 = insertAtEnd (head_2015, data2_2015);

    //cetak update list
    printList_2015 (head_2015);
    System.out.print("tambah 1 simpul ke data 4: ");
    int data3_2015 = 4;
    int pos_2015 = 4;
    head_2015 = insertPos (head_2015, pos_2015, data3_2015);
    //cetak update list
    printList_2015 (head_2015);
}
}

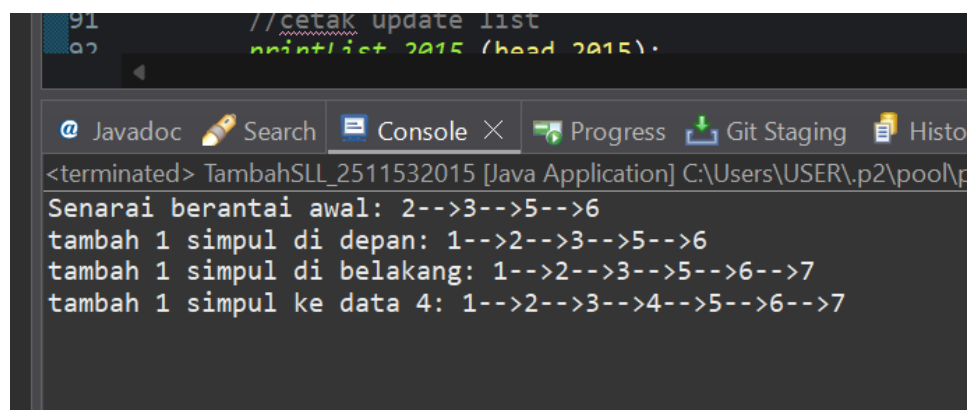
```

Kelas TambahSLL_2511532015 ini merupakan kumpulan method fungsi untuk melakukan operasi manipulasi pada struktur data Single Linked List. Semua fungsinya dirancang untuk menambahkan data baru ke dalam list, menampilkan isi list ke layar, dan membantu pembuatan node. Kelas ini tidak berdiri sendiri tapi ia bergantung pada kelas NodeSLL_2511532015 yang sudah dibuat sebelumnya. Method tersebut meliputi:

- Fungsi insertAtFront: menyisipkan node baru di posisi awal (head) list dengan kompleksitas waktu $O(1)$, karena hanya mengubah referensi head.

- insertAtEnd: menambahkan node di akhir list dengan melakukan traversing dari head hingga node terakhir (next = null) dengan kompleksitas waktu $O(n)$.
- GetNode(int data): bertugas membuat dan mengembalikan node baru dengan data tertentu yang berfungsi supaya kode di tempat lain lebih rapi (cukup panggil misal GetNode(5)) dan memudahkan jika ada logika tambahan saat membuat node.
- insertPos: menyisipkan node pada posisi spesifik yang ditentukan, dengan validasi batas posisi dan penanganan kasus khusus untuk penyisipan pertama.
- printList: mencetak semua data dalam list ke dalam layar dengan format data1-->data2-->data3-->data4
- main method atau program utama: bertugas untuk menguji dan menjalankan program beserta method-method nya. Alur eksekusinya dimulai dengan inisialisasi mnuua list berisi empat node (2, 3, 4, 5) dilanjutkan dengan serangkaian operasi menggunakan method yang telah dibuat. Setelahnya untuk menampilkan semua node dipanggil method printList_2015.

Program ini ketika dijalankan akan menghasilkan output:



```

//cetak update list
printList 2015 (head 2015).

<terminated> TambahSLL_2511532015 [Java Application] C:\Users\USER\p2\pool\p
Senarai berantai awal: 2-->3-->5-->6
tambah 1 simpul di depan: 1-->2-->3-->5-->6
tambah 1 simpul di belakang: 1-->2-->3-->5-->6-->7
tambah 1 simpul ke data 4: 1-->2-->3-->4-->5-->6-->7

```

Gambar 2.2.1: Hasil output program TambahSLL_2511532015

2.3 PencarianSLL

```

package pekan5_2511532015;

public class PencarianSLL_2511532015 {

```

```

static boolean searchKey (NodeSLL_2511532015 head_2015, int
key_2015) {
    NodeSLL_2511532015 curr_2015 = head_2015;
    while (curr_2015 != null) {
        if (curr_2015.data_2015 == key_2015)
            return true;
        curr_2015 = curr_2015.next_2015;
    }
    return false;
}

public static void traversal (NodeSLL_2511532015 head_2015) {
    //mulai dari head
    NodeSLL_2511532015 curr_2015 = head_2015;
    //telusuri sampai pointer null
    while (curr_2015 != null) {
        System.out.print(" "+ curr_2015.data_2015);
        curr_2015 = curr_2015.next_2015;
    }
    System.out.println();
}

public static void main (String[] args) {
    NodeSLL_2511532015 head_2015 = new NodeSLL_2511532015 (14);
    head_2015.next_2015 = new NodeSLL_2511532015 (21);
    head_2015.next_2015.next_2015 = new NodeSLL_2511532015
(30);
    head_2015.next_2015.next_2015.next_2015 = new
NodeSLL_2511532015 (10);
    System.out.print ("Penelusuran SLL: ");
    traversal (head_2015);
    //data yang akan dicari
    int key_2015 = 30;
    System.out.print ("cari data "+key_2015+ "= ");
    if (searchKey(head_2015, key_2015))
        System.out.println ("Ketemu");
    else
        System.out.println ("Tidak ada");
}
}

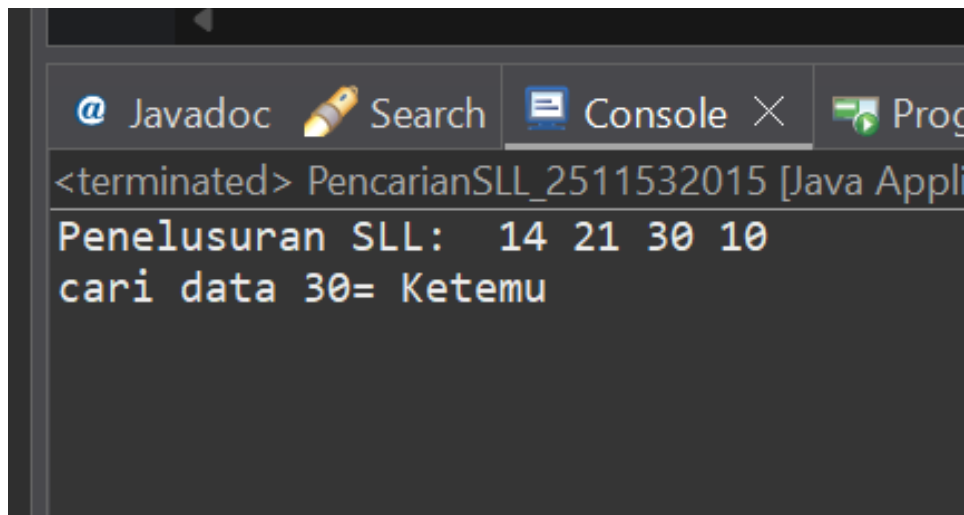
```

Program ini dibuat untuk mencari dan menampilkan data yang ada di dalam Single Linked List. Di dalam kode programnya terdapat beberapa method yaitu:

- searchKey: mencari apakah sebuah angka (key_2015) ada di dalam linked list yang akan true jika ketemu. Di sini terdapat perulangan while yang bekerja jika curr_2015 tidak sama dengan null. Jika data di node sama dengan key_2015 return true, jika bukan lanjut pindahkan curr_2015 ke node selanjutnya lewat pointer next. Dan kalau sudah sampai ke ujung tapi tidak ketemu return false.
- Traversal: menampilkan semua data dalam list ke layar.

- Main method: pertama membangun linked list dengan menambahkan beberapa angka ke dalam node, dan method traversal dipanggil untuk menampilkan daftar list. Dipanggil method searchKey melalui int key_2015 = 30 (angka yang akan di cari).

Program akan menampilkan output:



Gambar 2.2: Hasil Output program PencarianSLL_2511532015

2.4 HapusSLL

```
package pekan5_2511532015;

public class HapusSLL_2511532015 {
    //fungsi untuk menghapus head
    public static NodeSLL_2511532015 deleteHead
    (NodeSLL_2511532015 head_2015) {
        //jika SLL kosong
        if (head_2015 == null)
            return null;
        //pindahkan head ke node berikutnya
        head_2015 = head_2015.next_2015;
        //return head baru
        return head_2015;
    }
    //fungsi menghapus node terakhir SLL
    public static NodeSLL_2511532015 removeLastNode
    (NodeSLL_2511532015 head_2015) {
        //jika list kosong, return null
        if (head_2015 == null) {
            return null;
        }
        //jika list satu node, hapus node dan return null
        if (head_2015.next_2015 == null) {
            return null;
        }
        //temukan node terakhir ke dua
        NodeSLL_2511532015 secondLast = head_2015;
```

```

        while (secondLast.next_2015.next_2015 != null) {
            secondLast = secondLast.next_2015;
        }
        //hapus node terakhir
        secondLast.next_2015 = null;
        return head_2015;
    }
    //fungsi menghapus node di posisi tertentu
    public static NodeSLL_2511532015 deleteNode
(NodeSLL_2511532015 head_2015, int position_2015) {
        NodeSLL_2511532015 temp_2015 = head_2015;
        NodeSLL_2511532015 prev_2015 = null;
        //jika linked list null
        if (temp_2015 == null)
            return head_2015;
        //kasus 1: head dihapus
        if (position_2015 == 1) {
            head_2015 = temp_2015.next_2015;
            return head_2015;
        }
        //kasus 2: menghapus node di tengah
        //telusuri ke node yang dihapus
        for (int i = 1; temp_2015 != null && i < position_2015; i++)
        {
            prev_2015 = temp_2015;
            temp_2015 = temp_2015.next_2015;
        }

        //jika ditemukan, hapus node
        if (temp_2015 != null) {
            prev_2015.next_2015 = temp_2015.next_2015;
        } else {
            System.out.println ("Data tidak ada");
        }
        return head_2015;
    }
    //fungsi mencetak SLL
    public static void printList (NodeSLL_2511532015 head_2015)
    {
        NodeSLL_2511532015 curr_2015 = head_2015;
        while (curr_2015.next_2015 != null) {
            System.out.print (curr_2015.data_2015+"-->");
            curr_2015 = curr_2015.next_2015;
        }
        if (curr_2015.next_2015 == null) {
            System.out.print(curr_2015.data_2015);
        }
        System.out.println();
    }
}

//kelas main
public static void main (String[] args) {
    //buat SSL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
    NodeSLL_2511532015 head_2015 = new NodeSLL_2511532015 (1);
    head_2015.next_2015 = new NodeSLL_2511532015 (2);
    head_2015.next_2015.next_2015 = new NodeSLL_2511532015 (3);
    head_2015.next_2015.next_2015.next_2015 = new
NodeSLL_2511532015 (4);
}

```

```

        head_2015.next_2015.next_2015.next_2015.next_2015 = new
NodeSLL_2511532015 (5);

        head_2015.next_2015.next_2015.next_2015.next_2015.next_2015
= new NodeSLL_2511532015 (6);
        //cetak list awal
        System.out.println ("list awal: ");
        printList (head_2015);
        //hapus head
        head_2015 = deleteHead (head_2015);
        System.out.println ("List setelah head dihapus: ");
        printList(head_2015);

        //hapus node terakhir
        head_2015= removeLastNode (head_2015);
        System.out.println ("List setelah simpul terakhir di hapus:
");
        printList(head_2015);

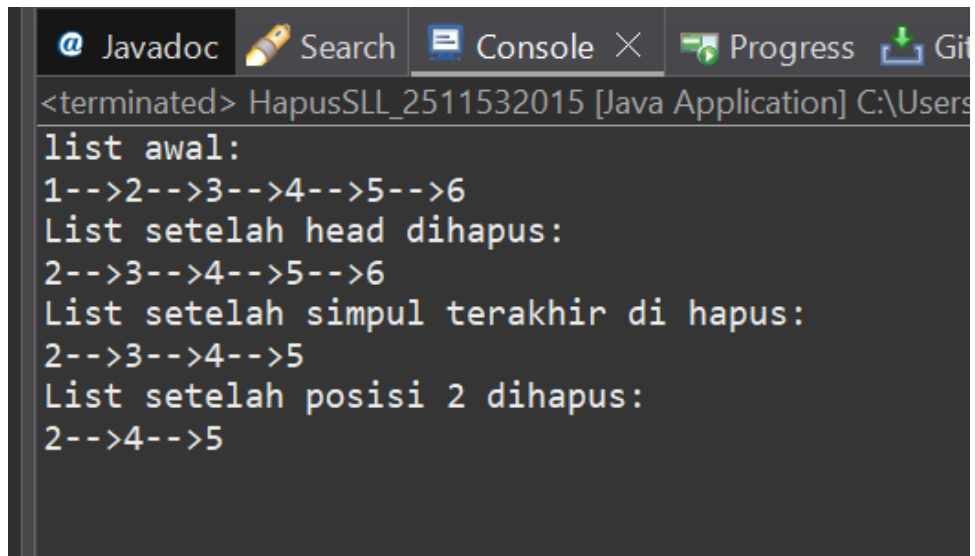
        //deleting node at position 2
        int position_2015 = 2;
        head_2015 = deleteNode (head_2015, position_2015);
        //print list after deletion
        System.out.println ("List setelah posisi 2 dihapus: ");
        printList(head_2015);
    }
}

```

Program ini berisi Kumpulan method untuk menghapus node dari Single Linked List.

- deleteHead: menghapus node paling depan, node lama otomatis terhapus
- removeLastNode: menghapus node paling belakang, cari node sebelum terakhir nextnya di null kan
- deleteNode: menghapus node di posisi yang ditentukan, rev.next = temp.next: sambungan dialihkan node targetnya dihapus
- printList: mencetak semua data list

Program akan menampilkan output:



```
<terminated> HapusSLL_2511532015 [Java Application] C:\Users
list awal:
1-->2-->3-->4-->5-->6
List setelah head dihapus:
2-->3-->4-->5-->6
List setelah simpul terakhir di hapus:
2-->3-->4-->5
List setelah posisi 2 dihapus:
2-->4-->5
```

Gambar 2.6: Hasil output program PenghapusanSLL_2511532015

BAB III

KESIMPULAN

3.1 Kesimpulan

Dari seluruh materi dan kode yang telah dipelajari, dapat disimpulkan bahwa struktur data Single Linked List adalah 12ar akit “node” atau simpul yang saling tersambung seperti rangkaian gerbong kereta, di mana setiap node menyimpan data dan penunjuk ke node berikutnya. Melalui kelas NodeSLL_2511532015 sebagai fondasi, kita dapat membangun berbagai operasi dasar seperti menambahkan data di depan, belakang, atau posisi tertentu (TambahSLL), mencari dan menampilkan isi list (PencarianSLL), hingga menghapus node sesuai kebutuhan (HapusSLL). Kunci utama dalam mengelola linked list adalah memahami cara kerja pointer next dan pentingnya memperbarui referensi head saat terjadi perubahan di awal list. Meskipun terdengar teknis, konsep ini sebenarnya logis dan mirip dengan 12ar akita menyusun atau memotong rantai kertas: selama sambungannya diatur dengan benar, data akan tetap tersusun rapi dan mudah dikelola. Dengan memahami dasar-dasar ini, kita sudah memiliki bekal penting untuk mengembangkan struktur data yang lebih dinamis dan efisien di masa mendatang.

DAFTAR PUSTAKA

- [1] Oracle, "The Java Tutorials: Data Structures," Oracle Corporation, 2023. [Online]. Available: <https://docs.oracle.com/javase/tutorial/collections/data-structures.html>. [Accessed 5 May 2026].
- [2] GreeksforGeeks, "Linked List Data Structure," GreeksforGeeks, 2023. [Online]. Available: <https://www.geeksforgeeks.org/data-structures/linked-list/>. [Accessed 5 May 2026].

LAPORAN TUGAS PEKAN 5

1. Kelas Pasien_2511532015

Program ini merupakan implementasi struktur data Single Linked List (SLL) yang diaplikasikan pada simulasi sistem antrian rumah sakit. Prinsip operasional utama yang digunakan adalah First In, First Out (FIFO), di mana pasien yang pertama kali mendaftar akan menjadi pasien pertama yang dipanggil. Berbeda dengan array yang memiliki ukuran statis, SLL pada program ini bersifat dinamis, seperti penambahan maupun penghapusan data dilakukan hanya dengan memanipulasi referensi pointer (next_2015) antar node, tanpa perlu menggeser elemen lain.

Kelas ini berfungsi sebagai cetakan node yang menyimpan atribur, dilengkapi dengan konstruktor untuk inisialisasi awal, serta selector (getter) dan mutator (setter).

```
package pekan5_2511532015;

public class Pasien_2511532015 {
    //atribut
    private String namaPasien_2015;
    private String keluhan_2015;
    private int nomorAntrian_2015;
    private Pasien_2511532015 next_2015;

    //konstruktor
    public Pasien_2511532015 (String namaPasien_2015, String
        Keluhan_2015, int nomorAntrian_2015){
        this.namaPasien_2015= namaPasien_2015;
        this.keluhan_2015 = keluhan_2015;
        this.nomorAntrian_2015 = nomorAntrian_2015;
        this.next_2015 = null;
    }
    //selektor (getter)
    public String getNamaPasien_2015(){
        return namaPasien_2015;
    }
    public String getKeluhan_2015(){
        return keluhan_2015;
    }
    public int getNomorAntrian_2015(){
        return nomorAntrian_2015;
    }
    public Pasien_2511532015 getNext_2015(){
        return next_2015;
    }
    //mutator (setter)
    public void setNamaPasien_2015(String namaPasien_2015){
        this.namaPasien_2015 = namaPasien_2015;
    }
}
```

```

    }
    public void setKeluhan_2015 (String keluhan_2015){
        this.keluhan_2015 = keluhan_2015;
    }
    public void setNomorAntrian_2015 (int nomorAntrian_2015){
        this.nomorAntrian_2015 = nomorAntrian_2015;
    }
    public void setNext_2015(Pasien_2511532015 next_2015){
        this.next_2015 = next_2015;
    }
}

```

2. RumahSakit_2511532015

Kelas ini mengelola keseluruhan melalui dua variable kunci yaitu head_2015 yang digunakan untuk referensi ke node pertama dan counter_2015 yang digunakan untuk penghitungan otomatis nomor antrian.

```

package pekan5_2511532015;

import java.util.Scanner;

public class RumahSakit_2511532015 {

    // === VARIABEL KUNCI ===
    private Pasien_2511532015 head_2015 = null;
    private int counter_2015 = 0;

    //method daftar pasien
    public void daftarkanPasien_2015(String nama, String keluhan)
    {
        Pasien_2511532015 pasienBaru_2015 = new
        Pasien_2511532015(nama, keluhan, counter_2015 + 1);
        if (head_2015 == null) { //langsung jadi head jika null
            head_2015 = pasienBaru_2015;
        } else {
            Pasien_2511532015 bantu_2015 = head_2015;
            while (bantu_2015.getNext_2015() != null) {
                bantu_2015 = bantu_2015.getNext_2015();
            }
            bantu_2015.setNext_2015(pasienBaru_2015);
        }

        counter_2015++; // naikkan counter setelah berhasil
        daftar
        System.out.println("Pasien berhasil didaftarkan! Nomor
        Antrian: " + pasienBaru_2015.getNomorAntrian_2015());
    }

    //method panggil pasien
    public void panggilPasien_2015() {
        if (head_2015 == null) {
            System.out.println("Antrian kosong. Tidak ada pasien
            yang bisa dipanggil.");
        }
    }
}

```

```

        return;
    }
    String nama_2015 = head_2015.getNamaPasien_2015();
    int nomor_2015 = head_2015.getNomorAntrian_2015();
    head_2015 = head_2015.getNext_2015();
    System.out.println("Pasien dipanggil: " + nama_2015 + "
(No. Antrian: " + nomor_2015 + ")");
}

//method tampilkan antrina
public void tampilkanAntrian_2015() {
    if (head_2015 == null) {
        System.out.println("Antrian kosong.");
        return;
    }

    System.out.println("\n=== Daftar Antrian ===");
    Pasien_2511532015 curr_2015 = head_2015;
    int posisi_2015 = 1;
    while (curr_2015 != null) {
        System.out.println(posisi_2015+"." + " +
curr_2015.getNamaPasien_2015() + " - " +
curr_2015.getKeluhan_2015());
        curr_2015 = curr_2015.getNext_2015();
        posisi_2015++;
    }
    System.out.println("=====");
}

//method cari pasien
public void cariPasien_2015(String nama) {
    if (head_2015 == null) {
        System.out.println("Antrian kosong.");
        return;
    }

    Pasien_2511532015 curr_2015 = head_2015;
    boolean ditemukan_2015 = false;
    while (curr_2015 != null) {
        if
(curr_2015.getNamaPasien_2015().equalsIgnoreCase(nama)) {
            System.out.println("Pasien ditemukan.");
            System.out.println("Nomor Antrian: " +
curr_2015.getNomorAntrian_2015());
            System.out.println("Keluhan: " +
curr_2015.getKeluhan_2015());
            ditemukan_2015 = true;
            break; // berhenti setelah ketemu pertama
        }
        curr_2015 = curr_2015.getNext_2015();
    }

    if (!ditemukan_2015) {
        System.out.println("Pasien dengan nama \"" + nama +
"\n" tidak ditemukan dalam antrian.");
    }
}

```

```

//method cek status antrian
public void cekStatusAntrian_2015() {
    if (head_2015 == null) {
        System.out.println("Antrian kosong.");
        return;
    }
    int total_2015 = 0;
    Pasien_2511532015 curr_2015 = head_2015;
    while (curr_2015 != null) {
        total_2015++;
        curr_2015 = curr_2015.getNext_2015();
    }
    System.out.println("\n=== Status Antrian ===");
    System.out.println("Total Pasien: " + total_2015);
    System.out.println("Pasien Terdepan: " +
        head_2015.getNamaPasien_2015() + " (No. " +
        head_2015.getNomorAntrian_2015() + ")");
    System.out.println("=====");
}

//menampilkan menu
private void tampilkanMenu_2015() {
    System.out.println("\n=== Antrian Rumah Sakit NIM: 2511532015 ===");
    System.out.println("1. Daftarkan Pasien (Insert at Tail)");
    System.out.println("2. Panggil Pasien (Delete Head)");
    System.out.println("3. Tampilkan Antrian (Display)");
    System.out.println("4. Cari Pasien (Search)");
    System.out.println("5. Cek Status Antrian");
    System.out.println("6. Keluar");
    System.out.print("Pilihan: ");
}

//main method
public static void main(String[] args) {
    Scanner scan_2015 = new Scanner(System.in);
    RumahSakit_2511532015 rs_2015 = new
    RumahSakit_2511532015();

    int pilihan_2015;

    do {
        rs_2015.tampilkanMenu_2015();
        pilihan_2015 = scan_2015.nextInt();
        scan_2015.nextLine();

        switch (pilihan_2015) {
            case 1:
                System.out.print("Masukkan Nama Pasien: ");
                String nama_2015 = scan_2015.nextLine();
                System.out.print("Masukkan Keluhan: ");
                String keluhan_2015 = scan_2015.nextLine();
                rs_2015.daftarkanPasien_2015(nama_2015,
                keluhan_2015);
                break;

```

```

        case 2:
            rs_2015.panggilPasien_2015();
            break;

        case 3:
            rs_2015.tampilkanAntrian_2015();
            break;

        case 4:
            System.out.print("Masukkan nama pasien yang
dicari: ");
            String cari_2015 = scan_2015.nextLine();
            rs_2015.cariPasien_2015(cari_2015);
            break;

        case 5:
            rs_2015.cekStatusAntrian_2015();
            break;

        case 6:
            System.out.println("Terima kasih telah
menggunakan sistem antrian.");
            break;

        default:
            System.out.println("Pilihan tidak valid.
Silakan pilih 1-6.");
    }
} while (pilihan_2015 != 6);

scan_2015.close();
}
}

```

3. Hasil output

```
=== Antrian Rumah Sakit NIM: 2511532015 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: khairun
Masukkan Keluhan: demam
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511532015 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 1
Masukkan Nama Pasien: fani
Masukkan Keluhan: pusing
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511532015 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 3
|
=== Daftar Antrian ===
1. khairun - null
2. fani - null
=====
```

Pada output di atas dapat dilihat method `daftarPasien` berjalan dengan menambahkan pasien sebanyak dua orang, dan di cek dengan memanggil method `tampilkanPasie_2015` (3).

```

=== Antrian Rumah Sakit NIM: 2511532015 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Pasien dipanggil: khairun (No. Antrian: 1)

```

Untuk pilihan menu 2, pasien dipanggil dan method panggilPasie_2015 akan menghapus pasien tersebut dari node.

```

=== Antrian Rumah Sakit NIM: 2511532015 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 2
Pasien dipanggil: khairun (No. Antrian: 1)

=== Antrian Rumah Sakit NIM: 2511532015 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 4
Masukkan nama pasien yang dicari: khairun
Pasien dengan nama "khairun" tidak ditemukan dalam antrian.

```

Untuk menu 4, method cariPasien, disini program meminta user untuk memasukkan nama pasien yang ingin dicari.

```

=== Antrian Rumah Sakit NIM: 2511532015 ===
1. Daftarkan Pasien (Insert at Tail)
2. Panggil Pasien (Delete Head)
3. Tampilkan Antrian (Display)
4. Cari Pasien (Search)
5. Cek Status Antrian
6. Keluar
Pilihan: 5

=== Status Antrian ===
Total Pasien: 1
Pasien Terdepan: fani (No. 2)
=====

```

Pilihan menu ke 5, cek status antrian, setelah tadi ditambahkan 2 orang pasien, dan 1 pasien dipanggil, maka antrian yang tinggal hanya 1 orang.

```
=== Antrian Rumah Sakit NIM: 2511532015 ===  
1. Daftarkan Pasien (Insert at Tail)  
2. Panggil Pasien (Delete Head)  
3. Tampilkan Antrian (Display)  
4. Cari Pasien (Search)  
5. Cek Status Antrian  
6. Keluar  
Pilihan: 6  
Terima kasih telah menggunakan sistem antrian.
```

Terakhir, menu ke 6 dipilih jika sudah selesai menggunakan program.